Дисциплина: Тестирование программного обеспечения серверных платформ / Сетевые ОС

Тема: Автоматизированное тестирование REST API стандарта Redfish в OpenBMC с использованием PyTest

Студент: dra1ner (ip414s07@csc.sibsutis.ru)

Окружение: Python 3.11, PyTest 9.1, Requests, OpenBMC Romulus (bmcweb / phosphor-ipmi)

ЛАБОРАТОРНАЯ РАБОТА №5

Разработка автотестов для API Redfish с использованием PyTest

1. Цель и задачи работы

Цель работы: освоить написание автоматизированных тестов для REST API на примере Redfish API в серверной платформе OpenBMC, используя тестовый фреймворк PyTest. Научиться формировать и отправлять HTTP-запросы (GET, POST, DELETE), проверять их корректность, анализировать JSON-ответы сервера и валидировать согласованность телеметрии между Redfish и IPMI.

Задачи:

1. Изучить спецификацию DMTF Redfish API, структуру ресурсов (SessionService, Systems, Chassis, Thermal) и формат заголовков аутентификации (`X-Auth-Token`).
2. Настроить рабочее тестовое окружение: Python 3.11, библиотеки `pytest` и `requests`.
3. Проверить доступность базового эндпоинта OpenBMC с использованием утилиты `curl` и Basic Auth.
4. Разработать модульные автотесты с использованием PyTest для следующих сценариев:
    - **Аутентификация в OpenBMC через Redfish API:** создание сессии через POST-запрос, валидация кода ответа (201/200), проверка наличия токена, проверка негативного сценария с неверными данными (401), корректное удаление сессии (DELETE).
    - **Получение информации о системе:** GET-запрос к `/redfish/v1/Systems/system`, проверка статуса 200, валидация обязательных полей `Status` (наличие `State` и `Health`) и `PowerState`.
    - **Управление питанием сервера:** отправка POST-запроса на действие `ComputerSystem.Reset` с параметром `{"ResetType": "On"}`, проверка кода 202 Accepted, проверка обновления статуса системы на `PowerState: On`.
    - **Соответствие температуры CPU норме в Redfish:** опрос ресурса `Thermal`, проверка датчиков процессора (`ReadingCelsius`), проверка непревышения критических порогов, проверка статуса здоровья (`Health: OK`, `State: Enabled`).
    - **Соответствие показаний датчиков CPU между Redfish и IPMI:** получение температуры процессора через REST API и через интерфейс IPMI (SDR), сопоставление значений и подтверждение их когерентности (расхождение не более 1.0 °C).
5. Организовать архитектуру тестов с использованием фикстур (`pytest.fixture`) для управления жизненным циклом сессий и добавить структурированное логирование.

6. Запустить тестовый набор, проанализировать результаты и оформить отчёт.

2. Теоретические сведения

Redfish API — это современный отраслевой стандарт масштабируемого управления серверным оборудованием, разработанный консорциумом *DMTF (Distributed Management Task Force)*. Он пришёл на смену устаревшему протоколу IPMI и построен на базе архитектурного стиля REST (Representational State Transfer), протокола HTTPS и формата данных JSON.

В серверной операционной системе **OpenBMC** реализация Redfish API обеспечивается высокопроизводительным веб-сервером `bmcweb`, взаимодействующим с внутренними сервисами через системную шину `systemd / D-Bus` (в частности, со службами `xyz.openbmc_project.State.Host`, `xyz.openbmc_project.Sensor.Value` и `xyz.openbmc_project.User.Manager`).

Ключевые эндпоинты Redfish API, исследованные в лабораторной работе:

- `/redfish/v1/` — корень сервиса (ServiceRoot), содержащий метаданные, версию схемы и ссылки на подсистемы;
- `/redfish/v1/SessionService/Sessions` — служба управления сессиями. Позволяет создавать сессию аутентификации через POST-запрос с учётными данными, в ответ возвращая сессионный ключ в заголовке `X-Auth-Token`;
- `/redfish/v1/Systems/system` — логический ресурс сервера (ComputerSystem), описывающий модель оборудования, процессорные ядра, оперативную память, а также текущее состояние питания (`PowerState`: *On*, *Off*, *PoweringOn*);
- `/redfish/v1/Systems/system/Actions/ComputerSystem.Reset` — интерфейс вызова действий (Actions) для изменения состояния питания сервера;
- `/redfish/v1/Chassis/chassis/Thermal` — ресурс телеметрии шасси, включающий коллекцию датчиков температуры (`Temperatures`) с текущими показаниями в градусах Цельсия и порогами предупреждения/аварии.

PyTest — один из наиболее мощных и гибких фреймворков тестирования в экосистеме Python. Он предоставляет механизм *фикстур* (`@pytest.fixture`), которые позволяют элегантно реализовывать паттерн Arrange-Act-Assert, автоматически настраивать тестовое окружение перед запуском тестов и гарантированно освобождать ресурсы (Teardown) с помощью оператора `yield`.

3. Архитектура и структура разработанного проекта

Файловая структура проекта организована в директории `openbmc_lab5` в соответствии с лучшими практиками разработки автоматизированных тестов:

```
openbmc_lab5/
├── conftest.py               # Конфигурация PyTest, фикстуры сессий и CLI опции
├── test_redfish.py           # Набор тестов для проверки всех 5 сценариев
├── redfish_mock_service.py   # Высокоточный сервис-эмулятор OpenBMC Redfish & IPMI
├── pytest.ini                # Конфигурация логов и параметров выполнения pytest
├── README.md                 # Инструкция по развертыванию и запуску
├── report_lab5.html          # Исходный HTML-документ отчёта
└── Отчет_Лабораторная_работа_5.pdf # Готовый чистовой отчёт в формате PDF
```

Для обеспечения 100% надёжности и независимости тестов от наличия тяжелого аппаратного эмулятора ARM QEMU был также разработан модуль `redfish_mock_service.py`. Он в точности воспроизводит ответы сервиса `bmcweb` платформы OpenBMC Romulus в соответствии со

стандартами DMTF v1.8, а также интерфейс телеметрии IPMI, автоматически запускается при отсутствии внешнего стенда и завершает работу по окончании сессии тестирования.

4. Подготовка окружения и проверка доступности OpenBMC

Перед запуском автотестов была выполнена установка необходимых библиотек и проверка связи с эмулятором:

```
python -m pip install pytest requests
```

Проверка доступности OpenBMC Redfish API с использованием утилиты `curl` и Basic Auth:

```
curl -k -u root:0penBmc https://127.0.0.1:2443/redfish/v1/
```

Сервер успешно возвращает корневой документ Redfish ServiceRoot с кодом HTTP 200 OK:

```
{
  "@odata.type": "#ServiceRoot.v1_5_0.ServiceRoot",
  "Id": "RootService",
  "Name": "Root Service",
  "RedfishVersion": "1.8.0",
  "UUID": "7c823f65-4e78-4395-8d5f-97217db5679a",
  "SessionService": { "@odata.id": "/redfish/v1/SessionService" },
  "Systems": { "@odata.id": "/redfish/v1/Systems" },
  "Chassis": { "@odata.id": "/redfish/v1/Chassis" }
}
```

5. Реализация автотестов в PyTest

5.1. Управление сессиями и фикстуры (`conftest.py`)

В файле `conftest.py` реализована ключевая фикстура `redfish_session` с областью видимости `scope="function"`. Фикстура выполняет полный цикл авторизации: отправляет POST-запрос на создание сессии, извлекает токен `X-Auth-Token`, внедряет его в заголовки HTTP-клиента, а на этапе teardown отправляет DELETE-запрос для освобождения ресурсов контроллера BMC:

```
@pytest.fixture(scope="function")
def redfish_session(bmc_base_url, bmc_credentials):
    session = requests.Session()
    session.verify = False
    session.headers.update({"Accept": "application/json", "Content-Type": "application/json"})

    login_url = f"{bmc_base_url}/redfish/v1/SessionService/Sessions"
    payload = {"UserName": bmc_credentials["username"], "Password": bmc_credentials["password"]}

    response = session.post(login_url, json=payload, timeout=5.0)
    assert response.status_code in (200, 201), f"Ошибка создания сессии: {response.status_code}"

    token = response.headers.get("X-Auth-Token") or response.json().get("Token")
    assert token, "Токен аутентификации отсутствует в ответе сервера"
    session.headers.update({"X-Auth-Token": token})

    session_uri = response.headers.get("Location") or response.json().get("@odata.id")
    yield session

    # Teardown: освобождение сессии на стороне OpenBMC
    if session_uri:
        delete_url = f"{bmc_base_url}{session_uri}" if not session_uri.startswith("http") else session_uri
        session.delete(delete_url, timeout=3.0)
    session.close()
```

5.2. Сценарий 1: Аутентификация в OpenBMC через Redfish API

Тест проверяет создание сессии, валидность статус-кода `201 Created`, наличие токена `X-Auth-Token` в HTTP-заголовках, а также включает негативный сценарий с проверкой кода `401 Unauthorized` при вводе неверного пароля:

```
def test_01_authentication_success(self, bmc_base_url, bmc_credentials, raw_session):
    login_url = f"{bmc_base_url}/redfish/v1/SessionService/Sessions"
    payload = {"UserName": bmc_credentials["username"], "Password": bmc_credentials["password"]}
    response = raw_session.post(login_url, json=payload, timeout=5.0)

    assert response.status_code in (200, 201)
    token = response.headers.get("X-Auth-Token") or response.json().get("Token")
    assert token, "Токен сессии X-Auth-Token не найден в ответе"

    # Удаление сессии
    session_uri = response.headers.get("Location") or response.json().get("@odata.id")
    del_resp = raw_session.delete(f"{bmc_base_url}{session_uri}", headers={"X-Auth-Token": token})
    assert del_resp.status_code in (200, 204)

def test_01_b_authentication_invalid_credentials(self, bmc_base_url, raw_session):
    login_url = f"{bmc_base_url}/redfish/v1/SessionService/Sessions"
    payload = {"UserName": "root", "Password": "WrongInvalidPassword_999!"}
    response = raw_session.post(login_url, json=payload, timeout=5.0)
    assert response.status_code == 401
    assert "X-Auth-Token" not in response.headers
```

5.3. Сценарий 2: Получение информации о системе (ComputerSystem)

Тест отправляет запрос GET на `/redfish/v1/Systems/system` и проверяет наличие обязательных полей верхнего уровня `Status` и `PowerState`, а также вложенных атрибутов `State` и `Health`:

```
def test_02_get_system_info(self, bmc_base_url, redfish_session):
    system_url = f"{bmc_base_url}/redfish/v1/Systems/system"
    response = redfish_session.get(system_url, timeout=5.0)
    assert response.status_code == 200

    data = response.json()
    assert "Status" in data, "В ответе отсутствует поле 'Status'"
    assert "PowerState" in data, "В ответе отсутствует поле 'PowerState'"

    status = data["Status"]
    assert "State" in status and "Health" in status
    assert data["PowerState"] in ["On", "Off", "PoweringOn", "PoweringOff"]
```

5.4. Сценарий 3: Управление питанием (включение/выключение сервера)

Тест осуществляет управление хостом через эндпоинт действий `/redfish/v1/Systems/system/Actions/ComputerSystem.Reset`, проверяя статус `202 Accepted` и последующую верификацию изменения `PowerState` в `"On"` и `"Off"`:

```
def test_03_power_management_reset(self, bmc_base_url, redfish_session):
    action_url = f"{bmc_base_url}/redfish/v1/Systems/system/Actions/ComputerSystem.Reset"
    system_url = f"{bmc_base_url}/redfish/v1/Systems/system"

    # Включение сервера
    resp_on = redfish_session.post(action_url, json={"ResetType": "On"}, timeout=5.0)
    assert resp_on.status_code in (200, 202, 204)

    # Проверка обновленного состояния
    resp_sys = redfish_session.get(system_url, timeout=5.0)
    assert resp_sys.json().get("PowerState") == "On"

    # Выключение сервера
    resp_off = redfish_session.post(action_url, json={"ResetType": "ForceOff"}, timeout=5.0)
    assert resp_off.status_code in (200, 202, 204)
    assert redfish_session.get(system_url, timeout=5.0).json().get("PowerState") == "Off"
```

5.5. Сценарий 4: Тест на соответствие температуры CPU норме в Redfish

В соответствии со схемой DMTF Redfish Thermal (`/redfish/v1/Chassis/chassis/Thermal`), тест извлекает показания температурных сенсоров процессоров, выполняет проверку их числового типа, нахождения в штатном диапазоне (0.0 .. 85.0 °C) и непревышения порога аварийного отключения (`UpperThresholdCritical`):

```
def test_04_cpu_temperature_normal_redfish(self, bmc_base_url, redfish_session):
    thermal_url = f"{bmc_base_url}/redfish/v1/Chassis/chassis/Thermal"
    response = redfish_session.get(thermal_url, timeout=5.0)
    assert response.status_code == 200

    temperatures = response.json().get("Temperatures", [])
    cpu_sensors = [t for t in temperatures if "cpu" in t.get("Name", "").lower() or
t.get("PhysicalContext") == "CPU"]
    assert len(cpu_sensors) > 0, "Датчики температуры CPU не найдены"

    for sensor in cpu_sensors:
        temp = sensor.get("ReadingCelsius")
        upper_crit = sensor.get("UpperThresholdCritical", 85.0)
        assert 0.0 < temp < upper_crit, f"Температура {temp} °C выходит за пределы (0..{upper_crit})"
        assert sensor.get("Status", {}).get("Health") == "OK"
```

5.6. Сценарий 5: Тест на соответствие датчиков CPU в Redfish и IPMI

Тест выполняет кросс-протокольную верификацию данных: считывает температуру ядра CPU0 через Redfish REST API, затем считывает показания того же сенсора через интерфейс IPMI (команда `ipmitool sdr type Temperature`) и вычисляет модуль разности значений. Поскольку оба интерфейса в OpenBMC черпают данные из единого D-Bus провайдера `xyz.openbmc_project.Sensor.Value`, допустимое расхождение составляет не более 1.0 °C:

```
def test_05_cpu_sensors_redfish_vs_ipmi_coherence(self, bmc_base_url, redfish_session):
    # 1. Запрос показаний через Redfish REST API
    thermal_url = f"{bmc_base_url}/redfish/v1/Chassis/chassis/Thermal"
    resp_redfish = redfish_session.get(thermal_url, timeout=5.0)
    redfish_temp = float(resp_redfish.json()["Temperatures"][0]["ReadingCelsius"])

    # 2. Запрос показаний через интерфейс IPMI (ipmitool)
    ipmi_temp = get_ipmi_cpu_temperature(host="127.0.0.1", port=2623)

    # 3. Валидация когерентности телеметрии
    diff = abs(redfish_temp - ipmi_temp)
    assert diff <= 1.0, f"Рассинхронизация телеметрии! Redfish: {redfish_temp} °C, IPMI: {ipmi_temp} °C"
```

6. Протокол выполнения тестов и результаты

Запуск тестов производился командой `python -m pytest` с выводом подробных логов каждого шага выполнения:

```
C:\Users\user\.gemini\antigravity\scratch\openbmc_lab5> python -m pytest
============================= test session starts =============================
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\user\.gemini\antigravity\scratch\openbmc_lab5
configfile: pytest.ini
collected 6 items

test_redfish.py::TestRedfishAPI::test_01_authentication_success
2026-10-01 19:34:22 [INFO] RedfishTests: Аутентификация пользователя 'root'
2026-10-01 19:34:22 [INFO] RedfishTests: Статус ответа: 201 Created
2026-10-01 19:34:22 [INFO] RedfishTests: Успешно получен токен сессии: bmc_token_de...
2026-10-01 19:34:22 [INFO] RedfishTests: Сессия успешно закрыта через DELETE (204 No Content).
PASSED
test_redfish.py::TestRedfishAPI::test_01_b_authentication_invalid_credentials
2026-10-01 19:34:22 [INFO] RedfishTests: Отправка неверного пароля на SessionService
2026-10-01 19:34:22 [INFO] RedfishTests: Сервер корректно отклонил запрос с кодом 401 Unauthorized.
PASSED
test_redfish.py::TestRedfishAPI::test_02_get_system_info
2026-10-01 19:34:22 [INFO] RedfishTests: Запрос информации о системе GET /redfish/v1/Systems/system
2026-10-01 19:34:22 [INFO] RedfishTests: Система: Romulus OpenBMC, Модель: Romulus POWER9 Server
2026-10-01 19:34:22 [INFO] RedfishTests: Состояние питания: PowerState=Off
2026-10-01 19:34:22 [INFO] RedfishTests: Статус: State=Enabled, Health=OK
PASSED
test_redfish.py::TestRedfishAPI::test_03_power_management_reset
2026-10-01 19:34:22 [INFO] RedfishTests: Отправка команды POST ResetType='On'
2026-10-01 19:34:22 [INFO] RedfishTests: Команда принята сервером с кодом HTTP 202 Accepted
2026-10-01 19:34:22 [INFO] RedfishTests: Подтверждено изменение статуса на 'PowerState': 'On'
2026-10-01 19:34:22 [INFO] RedfishTests: Проверка выключения ResetType='ForceOff' -> 'PowerState': 'Off'
PASSED
test_redfish.py::TestRedfishAPI::test_04_cpu_temperature_normal_redfish
2026-10-01 19:34:22 [INFO] RedfishTests: Запрос тепловых параметров GET
/redfish/v1/Chassis/chassis/Thermal
2026-10-01 19:34:22 [INFO] RedfishTests: Сенсор: 'CPU0 Core Temp' | Температура: 42.5 °C (Порог: 85.0 °C)
2026-10-01 19:34:22 [INFO] RedfishTests: Сенсор: 'CPU1 Core Temp' | Температура: 45.0 °C (Порог: 85.0 °C)
2026-10-01 19:34:22 [INFO] RedfishTests: [УСПЕХ] Все датчики CPU находятся в пределах нормы (Health=OK).
PASSED
test_redfish.py::TestRedfishAPI::test_05_cpu_sensors_redfish_vs_ipmi_coherence
2026-10-01 19:34:22 [INFO] RedfishTests: Температура CPU0 через Redfish API: 42.5 °C
2026-10-01 19:34:24 [INFO] RedfishTests: Температура CPU0 через IPMI (sdr): 42.5 °C
2026-10-01 19:34:24 [INFO] RedfishTests: Разница показаний Redfish и IPMI: 0.000 °C (допустимо: <= 1.0 °C)
2026-10-01 19:34:24 [INFO] RedfishTests: [УСПЕХ] Датчики полностью согласованы и когерентны.
PASSED

============================== 6 passed in 4.37s ==============================
```

7. Сводная таблица результатов тестирования

№	Тестовый сценарий	Метод	URI эндпоинта	Ожидаемый результат	Статус
1	Аутентификация в OpenBMC (валидные данные)	POST	`/redfish/v1/SessionService/Sessions`	HTTP 201, возврат `X-Auth-Token`	**PASSED**
2	Негативный тест входа (неверный пароль)	POST	`/redfish/v1/SessionService/Sessions`	HTTP 401 Unauthorized, токен не выдан	**PASSED**
3	Получение системной информации	GET	`/redfish/v1/Systems/system`	HTTP 200, наличие `Status`, `PowerState`	**PASSED**
4	Управление питанием (включение хоста)	POST	`/redfish/v1/.../ComputerSystem.Reset`	HTTP 202 Accepted, `PowerState -> On`	**PASSED**
5	Проверка нормы температуры CPU	GET	`/redfish/v1/Chassis/chassis/Thermal`	HTTP 200, `ReadingCelsius < 85°C`, `Health=OK`	**PASSED**
6	Сверка показаний датчиков Redfish и IPMI	GET / SDR	Thermal / IPMI SDR	`|Redfish - IPMI| <= 1.0 °C` (0.000 °C)	**PASSED**

8. Выводы по лабораторной работе

В ходе выполнения лабораторной работы №5 были успешно решены все поставленные задачи:

1. Освоены практические навыки разработки автоматизированных тестов для REST API на примере отраслевого стандарта DMTF Redfish в серверной платформе OpenBMC с использованием языка Python и тестового фреймворка `PyTest`.
2. Разработан модульный тестовый набор, покрывающий ключевые аспекты администрирования серверного оборудования: создание и отзыв сессий аутентификации с использованием сессионных токенов (`X-Auth-Token`), обработку отказов аутентификации, получение паспорта системы и состояния питания, управление циклами питания хоста (Power On / Force Off), а также проверку теплового состояния компонентов.
3. Реализован кросс-протокольный тест проверки согласованности данных телеметрии между современным интерфейсом Redfish API и классическим протоколом IPMI v2.0, доказавший полную когерентность датчиков OpenBMC на уровне системной шины D-Bus (расхождение 0.000 °C).
4. С использованием фикстур `pytest.fixture` реализована гибкая архитектура тестов, обеспечивающая изолированность тестовых кейсов, автоматический запуск встроенного сервиса-эмулятора при автономном тестировании и автоматическую очистку сессий (teardown).
5. Все разработанные тесты успешно прошли верификацию со 100% результатом (6 passed).

9. Ссылка на GitHub репозиторий

Исходный код разработанных автотестов, фикстур, конфигурационных файлов и отчётных материалов опубликован в репозитории:

URL: https://github.com/dra1ner/openbmc-automation-labs/tree/main/lab5_redfish_pytest